

Memory Planner and Correctness Checker for ML Pipelines.

Kurt Gu, Nels Martin, Vivek Sarkar

1 Introduction

1.1 Motivation

Training and post-train fine-tuning of machine learning models are now within reach using only a single consumer-grade GPU, but memory management within the training pipeline can be fragile and unpredictable when VRAM availability is constrained. This is compounded by the fact that the Python interpreter cannot detect static errors made within a training loop, and can break in the middle of execution due to memory limit or error in model configuration, resulting in potentially costly runtime crashes.

Since many such training routines can take a long time (for example, GPT-2 pre-training can be done in around 24 hours using *llm.c*[2].), such an error incurs a serious penalty on training progress and compute cost. Therefore, it would be useful to have a memory scheduler that can estimate memory usage in each step of the script, taking into consideration factors such as dataset size, model dimensionality, and intermediate activation size. The goal of the scheduler would be to:

- Perform necessary evictions automatically to avoid out-of-memory errors, by planning out when to free which VRAM-resident item.
- Keep as many pieces of relevant data in memory as possible to facilitate faster training and more efficient data movement.

As shown in prior work [1], PyTorch models can be statically analyzed before execution to ensure correctness of tensor dimensions. Similarly, we posit that use-dependencies between VRAM-resident items such as tensors and datasets can also be analyzed and planned ahead of time. Our proposal is to use solvers to perform these analyses and planning before the training script is executed, and report detectable conflicts/errors.

1.2 Example Problem

To illustrate more concretely the use case of such a scheduler, we include the following scenario, modeled after another ongoing research project:

Given a VRAM budget of 24GB, a 12GB dataset D_1 where each datapoint is 1KB, and initially a convolutional neural net (CNN) A of 3 million parameters with dense activations. After training concludes, we also run visualizations using A , a subset of the dataset, as well as outputs from A . Afterwards, a 7GB large language model B is loaded and partially modified by model A . Finally, B is finetuned using a different dataset D_2 , during which additional memory needs to be allocated for activations and the KV -cache.

This pipeline could be executed within the VRAM budget if the dataset is freed after visualization, and A after modifying B , but would incur out-of-memory errors in a naive sequential execution. Furthermore, while it would reduce memory usage, it would be suboptimal to, for example, save and reload A when bringing B into VRAM. A programmer could certainly manually implement an eviction policy that fits the VRAM budget by profiling the memory usage of each component, but would need to do so on a case-by-case basis for different training routines. The aim of our project is to build a solver-aided memory scheduler that could estimate memory usage statically and plan run-time evictions automatically, alleviating the memory management burden on programmers.

2 Overview

2.1 Related Work

The increase in the popularity and size of deep learning (DL) models has inspired research on how to train DL models on devices with limited memory. Models can either be analyzed before, during, or after runtime, and may or may not be adjusted to fit within a given memory constraint. In the category of run-time adjustment, the Dynamic Tensor Rematerialization Algorithm (DTR) introduced by Kirisame et al. in 2021 [3] is an online algorithm that checkpoints deep learning models during training, dynamically freeing activations from memory. DTR frees memory according to heuristics over data gathered during runtime, and therefore requires no prior knowledge of the model or program.

There is also a precedent for using an SMT solver to guarantee certain characteristics of a model statically. As mentioned above, in the paper “A Static Analyzer for Detecting Tensor Shape Errors in Deep Neural Network Training Code” [1], the authors introduce a static analyzer that traces all possible paths of a PyTorch program and collects constraints over the interrelated sizes of the tensors the program creates.

Some programs statically analyze programs for memory safety. Rockmate [4] is another rematerialization tool introduced in 2023 that, given a PyTorch model and a memory constraint, produces a new model that can be trained without exceeding the memory constraint. It does so by analyzing the data and computational dependencies of the program, and then rewriting the program into complex “blocks”, each of which may be computed using different methods adapted to their respective characteristics.

Our proposed tool is similar to some of the above solutions in that it will statically determine the memory safety of a program and rely on calls to an SMT solver, but different in scope. While the above methods are concerned with individual model training runs, we are concerned with dependencies *between* model training runs and evaluations.

2.2 Implementation

For this project, we designed a domain-specific language (DSL) to express training pipelines and model architectures of ML workloads, and used Racket and Rosette to analyze their parameterizations and implicit dependencies, resulting in a proof-of-concept model correctness checker and memory planner.

The implementation is structured into two modules. The `syntax.rkt` module defines the DSL primitives: struct definitions for layers, models, datasets, training jobs, and pipeline configurations; constructors for each layer type with appropriate parameter validation; and macros (`train`, `pipeline`, `define-component`) that parse the DSL syntax into internal representations.

The `validate.rkt` module implements the analysis pipeline:

1. **Job Analysis:** Each training job is analyzed to extract models and datasets, validate layer dimensions, count parameters, and compute memory estimates.
2. **Component Identification:** Unique models are identified across all jobs using Racket’s `eq?` comparison on model structs, and assigned numeric IDs.
3. **Liveness Analysis:** For each component, the first and last job indices that use it are computed to determine live intervals.
4. **Constraint Generation:** Symbolic batch size variables are created for each job, and constraints over linear integer arithmetic are assembled.
5. **Optimization:** Rosette’s `optimize` procedure maximizes total batch size subject to the constraints.
6. **Plan Generation:** The solver output is interpreted to produce a concrete execution plan with load/evict/cache operations.

The constraints encode memory requirements as linear expressions over batch sizes, which fall within the theory of linear integer arithmetic (LIA).

3 Algorithms and Logical Encodings

3.1 The DSL and its syntax

Our DSL enables programmers to represent a subset of the models, datasets, and training pipelines that comprise PyTorch programs. The implementation consists of two Racket modules: `syntax.rkt` defines the DSL primitives and parsing logic, while `validate.rkt` implements the analysis and solver integration.

A model is defined by providing an optional bit-width specification (defaulting to 32 bits) followed by a sequence of layer definitions. The supported layer types are:

- `(linear in-dim out-dim [activation])`: A fully-connected layer
- `(conv in-ch out-ch kernel [activation] #:stride s #:padding p)`: A 2D convolutional layer
- `(maxpool kernel [stride])`: A max-pooling layer
- `(flatten)`: A flattening layer that reshapes multi-dimensional activations to 1D

Datasets are defined by specifying a name, the number of examples, an optional bit-width, and the dimensions of each example. For instance, a model with 16-bit activations containing a convolutional layer with 3 input channels, 32 output channels, a 3×3 kernel, and ReLU activation, followed by max-pooling, flattening, and a final linear layer, trained on CIFAR-style data, would be encoded as:

```
(train (model (bits 16) (conv 3 32 3 relu) (maxpool 2) (flatten) (linear 7200 10))
  on (dataset "CIFAR" 2000 (bits 16) 3 32 32))
```

Models can also be defined with `define-component`, which binds a model to a name and assigns it a unique identifier via `gensym`. This enables component reuse across multiple training jobs:

```
(define-component encoder (bits 32) (linear 500 250 relu))
(define-component decoder (bits 32) (linear 250 500 relu))
(define-component generator (bits 32) (linear 250 250 relu))

(pipeline
  (train encoder decoder on (dataset "AutoEnc" 100 (bits 32) 500))
  (train encoder generator on (dataset "GenTrain" 100 (bits 32) 500))
  (train generator decoder on (dataset "Finetune" 100 (bits 32) 250))
  with (vram 4.5))
```

The `pipeline` macro collects training jobs and configuration parameters, then invokes the solver. The VRAM limit is specified in megabytes.

3.2 Correctness Checking

The DSL performs static dimension checking by propagating tensor shapes through the network architecture. For each layer type, the validator computes output dimensions from input dimensions:

For a **linear layer** with input dimension d_{in} and output dimension d_{out} , the validator checks that the current shape is 1-dimensional with size equal to d_{in} , and produces output shape (d_{out}).

For a **convolutional layer** with c_{in} input channels, c_{out} output channels, kernel size k , stride s , and padding p , given input shape (c, h, w) , the validator checks $c = c_{in}$ and computes:

$$h' = \left\lfloor \frac{h + 2p - k}{s} \right\rfloor + 1, \quad w' = \left\lfloor \frac{w + 2p - k}{s} \right\rfloor + 1$$

producing output shape (c_{out}, h', w') .

For a **maxpool layer** with kernel size k and stride s , given input (c, h, w) :

$$h' = \left\lfloor \frac{h}{s} \right\rfloor, \quad w' = \left\lfloor \frac{w}{s} \right\rfloor$$

A **flatten layer** maps shape $(d_1, d_2, \dots, d_n)$ to $(\prod_i d_i)$.

If any dimension mismatch is detected, the validator reports the error with the specific job, model, and layer indices. For example:

```
(pipeline
  (train (model (bits 32) (linear 784 128 relu) (linear 127 10))
    on (dataset "MNIST-A" 1000 (bits 32) 784))
  with (vram 100))
```

This triggers an error because the first linear layer outputs dimension 128, but the second layer expects input dimension 127.

3.3 Memory Estimation

The memory footprint is estimated by counting parameters and activations. For each layer, the parameter count is:

$$\begin{aligned} \text{Linear}(d_{in}, d_{out}) &: d_{in} \cdot d_{out} + d_{out} \\ \text{Conv}(c_{in}, c_{out}, k) &: c_{in} \cdot c_{out} \cdot k^2 + c_{out} \end{aligned}$$

The static memory for a model accounts for optimizer state. Assuming an Adam-style optimizer, each parameter requires storage for the weight, gradient, and two momentum terms, yielding a $4\times$ multiplier:

$$\text{StaticBytes}(M) = \sum_{\ell \in M} \text{params}(\ell) \cdot \frac{\text{bit-width}}{8} \cdot 4$$

Dynamic memory per sample includes activations (computed by summing output dimensions across layers, multiplied by 4 bytes and a factor of 2 for forward/backward passes) and input data (product of input dimensions times data bit-width).

3.4 Liveness Analysis and Caching

Before constraint generation, the solver performs liveness analysis on model components. For each unique model M , it computes the *live interval* $[j_{first}, j_{last}]$ where j_{first} and j_{last} are the indices of the first and last jobs that use M .

This information enables three memory management strategies:

- **Loading:** A model is loaded into VRAM when first needed.
- **Freeing:** A model is freed after its last use (lifespan ended).
- **Caching:** If memory permits, idle models within their live interval are cached to avoid reload costs.

3.5 Constraint Formulation and Optimization

For each job j in the pipeline, we introduce a symbolic integer variable b_j representing the batch size. The constraints encode:

$$b_j \geq 1 \tag{1}$$

$$b_j \leq |D_j| \quad (\text{dataset size}) \tag{2}$$

$$\text{StaticMem}_j + \text{DynMem}_j(b_j) \leq \text{VRAM}_{\text{limit}} \tag{3}$$

where StaticMem_j is the sum of static memory for all models required by job j , and $\text{DynMem}_j(b_j) = b_j \cdot (\text{act_bytes_per_sample} + \text{input_bytes_per_sample})$.

These constraints are passed to Rosette’s **optimize** procedure with the objective of maximizing $\sum_j b_j$. Rosette translates this into an SMT query over the theory of linear integer arithmetic (LIA).

If the constraints are satisfiable, the solver returns concrete batch sizes for each job. The planner then simulates execution, tracking the resident set of models and generating a detailed execution plan that specifies which models to load, hold, cache, evict, or free at each step.

4 Summary of Results

We evaluated the DSL and solver on a suite of test cases that exercise different aspects of the system.

4.1 Test Cases

Test 1 (Simple Sequential Pipeline): Two independent models trained sequentially on MNIST-style data with a 1024 MB budget. The solver successfully finds feasible batch sizes for both jobs.

Test 2 (Tight VRAM Constraint): A convolutional model on CIFAR-style data followed by a linear model, with only 50 MB available. This tests the solver’s ability to find small but feasible batch sizes under pressure.

Test 3 (Impossible Pipeline): Two linear models each requiring approximately 16 MB of static memory, but with only 20 MB total VRAM. The solver correctly reports **FAILED (OOM)** since even batch size 1 exceeds the budget.

Test 4 (Auto-Partitioning): Three sequential jobs with independent models, demonstrating that components are loaded and freed at job boundaries without unnecessary caching.

Test 5 (Fine-Grained Component Liveness): The encoder-decoder-generator example from Section 3. The encoder is used in jobs 1 and 2, the decoder in jobs 1 and 3, and the generator in jobs 2 and 3. With 4.5 MB VRAM, the solver produces a plan that:

- Job 1: Loads encoder and decoder
- Job 2: Holds encoder, evicts decoder (to make room), loads generator
- Job 3: Frees encoder (lifespan ended), loads decoder, holds generator

Test 6 (Extended Config): A large model used in jobs 1 and 3, with a small model in job 2. The solver caches the large model during job 2 if memory permits, avoiding a reload.

4.2 Observations

The solver correctly handles dimension mismatches, reporting precise error locations (job index, model index, layer index). The optimization successfully maximizes total batch size across jobs while respecting VRAM constraints.

The liveness-aware caching strategy avoids unnecessary evictions when memory is available, and the greedy cache-filling heuristic works well for the tested pipelines, though it does not guarantee global optimality.

4.3 Caveats and Future Work

- The current DSL assumes all models in a training job have their parameters updated. A future version could distinguish between trainable and frozen components, reducing the memory multiplier for frozen weights.
- The dataset specification requires explicit dimensions and bit-width. Integration with standard dataset libraries could automate this.
- The DSL requires adoption of a new syntax. A more practical approach would be to build a Python frontend that extracts the relevant information from PyTorch code, either through static analysis or instrumented execution.
- The memory model assumes Adam-style optimizers ($4\times$ parameter memory). Supporting other optimizers (SGD, AdaGrad) would require parameterizing the multiplier.
- The current implementation does not model gradient checkpointing or activation recomputation, which are common techniques for reducing memory usage.

5 Teamwork

6 Course Topics

As stated, the main aspect of this project using SAT/SMT solvers involved verifying that there was enough VRAM to train all the models in a pipeline, as well as to fit the datasets. The part of the course where we talked about SMT theories (specifically LIA) and how one uses them together in Rosette. Secondly, we also used solvers to maximize the batch size - the most useful part of the course for this was in Lecture 5, when we talked about pseudo-boolean constraints.

References

- [1] Ho Young Jhoo, Sehoon Kim, Woosung Song, Kyuyeon Park, DongKwon Lee, and Kwangkeun Yi. A static analyzer for detecting tensor shape errors in deep neural network training code. In *Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings*, ICSE '22, page 337–338, New York, NY, USA, 2022. Association for Computing Machinery.
- [2] Andrej Karpathy. llm.c: Llm training in simple, raw c/cuda. <https://github.com/karpathy/llm.c>, 2024. Accessed: December 7, 2025.
- [3] Marisa Kirisame, Steven Lyubomirsky, Altan Haan, Jennifer Brennan, Mike He, Jared Roesch, Tianqi Chen, and Zachary Tatlock. Dynamic tensor rematerialization, 2021.
- [4] Xunyi Zhao, Théotime Le Hellard, Lionel Eyraud, Julia Gusak, and Olivier Beaumont. Rockmate: an efficient, fast, automatic and generic tool for re-materialization in pytorch, 2023.